DESIGN AND FPGA IMPLEMENTATION OF AN EFFICIENT PARTIAL PRODUCT GENERATOR AND ACCUMULATION UNIT FOR MULTIPLIERS

A Project Work Report under ECE-327 (Project-1)

Submitted in partial fulfilment of the Requirements for the Award degree of

Bachelor of Technology

In

Electronics & Communications Engineering

Submitted by

GROUP NO – 16

Group Members –

Chinmay Kushwah (2311401170)

Aditya Raghunath Raut (2311401264)

MD. Hasan Usmani (2311401173)

Mohd. Wasim Akhtar (2311401269)

Under the Guidance of

Dr. Laxmi Pradhan Kumre

April – 2026

Department of Electronics & Communication Engineering

Maulana Azad National Institute of Technology, Bhopal

Maulana Azad National Institute of Technology, Bhopal

DECLARATION BY CANDIDATES

We hereby declare that the Report of the Project Work entitled "Design of an Efficient Partial Product Generator and Accumulation Unit for Multipliers" which is being submitted to the **Department of Electronics & Communication Engineering (ECE), MANIT Bhopal**, in the partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in the **Department of ECE**, is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any University or Institution for the award of any degree.

S. No.	Scholar No.	Name of Students	Signature of Students
1	2311401170	Chinmay Kushwah	
2	2311401264	Aditya Raghunath Raut	
3	2311401173	MD. Hasan Usmani	
4	2311401269	Mohd. Wasim Akhtar	

CERTIFICATE OF APPROVAL

It is certified that the work contained in this report titled "**Design of an Efficient Partial Product Generator and Accumulation Unit for Multipliers**" is the work done by the students and has been carried out under my supervision.

(Signature of the Supervisor with Date)

Name of Supervisor: Dr. Laxmi Pradhan Kumre

Department of Electronics and Communication Engineering

Maulana Azad National Institute of Technology, Bhopal

ABSTRACT

Digital multipliers are fundamental arithmetic components in high-performance computing systems, determining critical metrics such as speed, area utilization, and power consumption. Our project presents the design, simulation, and FPGA-based implementation of three 8-bit signed multiplier architectures: the Array (shift-and-add) multiplier, the Booth Radix-2 multiplier, and the Modified Booth Radix-4 multiplier. All architectures are described using Verilog Hardware Description Language (HDL) and implemented on the Xilinx Artix-7 FPGA (Basys-3 board) using the Vivado Design Suite.

A modular, non-pipelined, structural design methodology is adopted throughout, wherein the complete multiplier is built by interconnecting individually designed submodules including a Booth Encoder, Partial Product Generator (PPG), Carry-Save Adder (CSA), and Final Adder. This approach enables clear architectural comparison by isolating the contribution of each stage. All three designs are synthesized and implemented under identical constraints to ensure a fair comparative evaluation.

Results reveal distinct trade-offs among the three architectures. The Array multiplier achieves the smallest area footprint with only 65 LUTs, while Booth Radix-4 closely follows with 70 LUTs. From a timing perspective, the Array multiplier exhibits the largest Worst Negative Slack (WNS = 3.332 ns), indicating ample timing margin, while Booth Radix-2 presents the tightest timing (WNS = 2.326 ns). Power analysis shows that static device power dominates across all designs; however, Booth Radix-2 records the lowest dynamic power at 1 mW, whereas Booth Radix-4 unexpectedly exhibits the highest dynamic power at 5 mW due to the complexity of its modified encoder logic. These findings highlight that on FPGA platforms, technology mapping characteristics significantly influence implementation outcomes beyond purely algorithmic considerations.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and supervisor, **Dr. Laxmi Pradhan Kumre**, Assistant Professor, Department of Electronics and Communication Engineering, MANIT Bhopal, for her invaluable guidance, constant support, and constructive feedback throughout the course of this project. Her technical expertise and encouragement were instrumental in shaping the direction and quality of our work.

We extend our sincere thanks to **Dr. Dheeraj K. Agrawal**, Head of the Department of Electronics and Communication Engineering, for providing access to the necessary departmental resources and laboratory infrastructure required for FPGA implementation and testing.

We are also grateful to **Dr. Karunesh Kumar Shukla,** Director, MANIT Bhopal, for providing an academic environment conducive to research and innovation. We acknowledge the support of all faculty members, laboratory staff, and technical assistants of the ECE Department who assisted us throughout this project.

Finally, we thank our families and batchmates for their unwavering moral support and encouragement during the course of this work.

Sincerely,
Chinmay Kushwah (2311401170)
Aditya Raghunath Raut (2311401264)
MD. Hasan Usmani (2311401173)
Mohd. Wasim Akhtar (2311401269)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol / Abbreviation	Full Form / Meaning
HDL	Hardware Description Language
FPGA	Field-Programmable Gate Array
PPG	Partial Product Generator
CSA	Carry-Save Adder
LUT	Look-Up Table
WHS	Worst Hold Slack
WNS	Worst Negative Slack
MBA	Modified Booth Algorithm
DSP	Digital Signal Processing
ALU	Arithmetic Logic Unit
RTL	Register Transfer Level
DXA	Twenty-Fifths of an Inch (Word unit)
n	Bit-width of operands
M	Multiplicand
Q	Multiplier
A	Accumulator register
2's complement	Signed binary number representation

1. INTRODUCTION

Multiplication is one of the most computationally arithmetic operations performed in digital systems. Multipliers are essential components in a wide spectrum of applications ranging from Digital Signal Processing (DSP) and image processing to cryptography and neural network accelerators. The performance of these systems is critically dependent on the speed, area efficiency, and power consumption of the underlying multiplier hardware. Consequently, the design and optimization of multiplier architectures have been a long-standing area of research in digital VLSI design.

In our project, three 8-bit signed multiplier architectures are designed, simulated, and physically implemented on an FPGA: the Array (shift-and-add) multiplier, the Booth Radix-2 multiplier, and the Modified Booth Radix-4 multiplier. All designs are described using Verilog HDL and evaluated on the Xilinx Basys-3 development board (Artix-7 FPGA) using the Vivado Design Suite, enabling a fair, platform-consistent comparison. As shown in Fig. 1.1, the workflow progresses from Verilog HDL coding through simulation, synthesis, place and route, to final implementation on the FPGA.

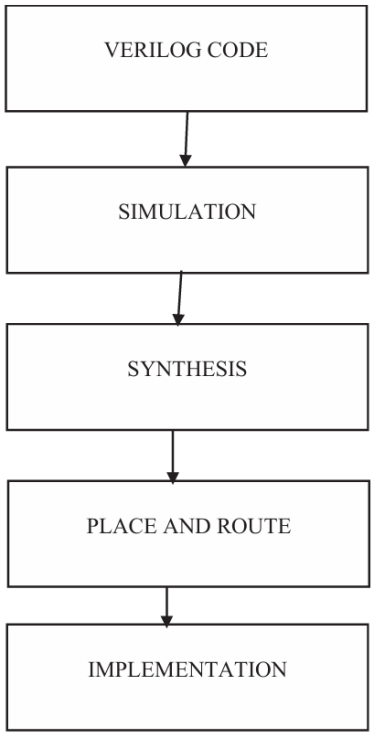

Figure 1.1: Workflow Diagram (Verilog to Implementation)

1.1 BACKGROUND AND MOTIVATION

Multipliers occupy significant area and contribute substantially to the power budget and critical path delay in digital designs. The fundamental trade-off between area and speed has motivated the development of numerous multiplier architectures. Simple shift-and-add multipliers are area-efficient but generate a large number of partial products, leading to longer accumulation times. Booth's algorithm addresses this by recoding the multiplier in a signed digit representation, thereby reducing the number of partial products. The Modified Booth Algorithm (Radix-4) extends this further by halving the partial product count. Despite these theoretical advantages, FPGA technology mapping introduces additional nuances that make practical evaluation essential — architectural benefits proven at the gate level do not always translate directly to FPGA synthesis outcomes.

1.2 OBJECTIVES

The specific objectives of our project are as follows:

- To implement three distinct 8-bit signed multiplier architectures — Array, Booth Radix-2, and Booth Radix-4 — in Verilog HDL using a structural and modular design approach.
- To simulate all three designs using the Vivado (Xilinx) simulator and verify functional correctness through testbench-driven waveform analysis.
- To synthesize and physically implement all designs on the Xilinx Artix-7 FPGA (Basys-3) under identical timing and area constraints.
- To compare the three architectures on three key metrics: area utilization (LUT count), timing performance (WHS and WNS), and power consumption (static and dynamic).
- To identify the most suitable architecture for area-constrained, power-critical, and speed-balanced design scenarios.

1.3 SCOPE OF THE PROJECT

The scope of this project is limited to 8-bit signed multiplication using 2's complement representation. Pipelining is intentionally excluded to enable direct architectural comparison without throughput-related confounding factors. The evaluation platform is FPGA, and all results are obtained from Vivado's post-implementation reports. ASIC implementation and gate-level power estimation using switching activity files (SAIF) are outside the scope of this work but are proposed as future extensions.

2. LITERATURE SURVEY

2.1 EXISTING MULTIPLIER ARCHITECTURES

The earliest digital multipliers employed the idea of shifting and adding the multiplicand corresponding to each '1' in the multiplier bits. Although quite simple, this approach leads to the creation of n partial products in the case of an n-bit multiplier. Each of these partial products needs to be added, leading to considerable delays. Booth encoding [1], which was suggested by Andrew D. Booth in 1951, helped overcome this problem by coding the multiplier as signed digits, permitting not only addition but also subtraction. The result of such coding is the absence of runs of '1's in the multiplier and an average reduction to n/2 additions in the case of Radix-2.

The Modified Booth Algorithm (MBA), also referred to as Radix-4 Booth encoding [2], further halved the partial product count by examining overlapping groups of three multiplier bits at a time. This results in n/2 partial products, each belonging to the set {0, ±M, ±2M}, where M is the multiplicand. The Carry-Save Adder (CSA) [3] provides an efficient mechanism for accumulating multiple partial products simultaneously, offering speed advantages over sequential ripple-carry addition.

For FPGA implementations, several studies have highlighted the discrepancy between theoretical gate-level models and practical synthesis outcomes. Tam et al. [5] noted that FPGA LUT-based technology mapping absorbs some encoding logic differently from ASIC standard cells, making direct comparisons valuable. Shambhavi et al. [6] demonstrated that for small bit-widths, array multipliers can outperform Booth variants in area due to simpler logic structures that map well onto LUT primitives.

2.2 CHALLENGES IN HARDWARE MULTIPLIER DESIGN

The primary challenge in multiplier design is the inherent conflict between area and speed. Optimizing for speed typically requires wider adder trees and more parallel logic, increasing area and power. Conversely, area-minimized designs often introduce longer critical paths, degrading timing performance. On FPGA platforms, additional challenges arise from the fixed nature of LUT-based logic resources. Booth encoding logic, particularly for Radix-4, introduces extra routing complexity that the synthesizer may not be able to fully absorb into carry chains, leading to higher-than-expected LUT counts [7].

Power estimation on FPGAs is further complicated by the dominance of static device power, which is a fixed characteristic of the FPGA process technology and accounts for 96–99% of total on-chip power in low-activity designs. This masks differences in dynamic switching power between architectures. Accurate power comparison requires simulation-based switching activity annotation, which is beyond the scope of this work. Additionally, signed number handling using 2's complement introduces sign-extension requirements that add overhead to partial product generation, particularly for the array multiplier [8].

3. MATERIALS AND METHODOLOGY

3.1 TOOLS AND MATERIALS

The following hardware and software tools were used for the design, simulation, and implementation of the three multiplier architectures:

- Verilog HDL: The primary hardware description language used for all design and submodule implementations, employing structural and behavioural modeling styles.
- Xilinx Vivado Design Suite : Used for RTL elaboration, logic synthesis, place and route, timing analysis, and power estimation.
- Xilinx Basys-3 Development Board: The target FPGA platform, featuring an Artix-7 (XC7A35T) device, used for physical implementation and on-board verification.
- Vivado XSim Simulator: The built-in simulation tool used for functional verification of all three designs through waveform analysis.

3.2 METHODOLOGY

A modular, non-pipelined, structural design methodology is adopted throughout this project. Each multiplier architecture is decomposed into independently designed and verified submodules, which are then hierarchically interconnected to form the complete multiplier. This approach separates the functional responsibilities of encoding, partial product generation, accumulation, and final addition, enabling clear architectural comparison and independent optimization of each stage.

3.2.1 Partial Product Generation (PPG)

Partial products are generated using the input operands after Booth encoding. For the Array multiplier, partial products are computed directly as bitwise AND operations between the multiplicand and each bit of the multiplier. For the Booth multipliers, the PPG generates partial products based on the encoded multiplier values: {0, +M, -M} for Radix-2 and {0, +M, -M, +2M, -2M} for Radix-4. Negation is implemented using 2's complement, and multiplication by 2 is performed by a one-bit left shift. As shown in Fig.

3.1, the array hardware organizes these partial products in a grid pattern for subsequent accumulation.

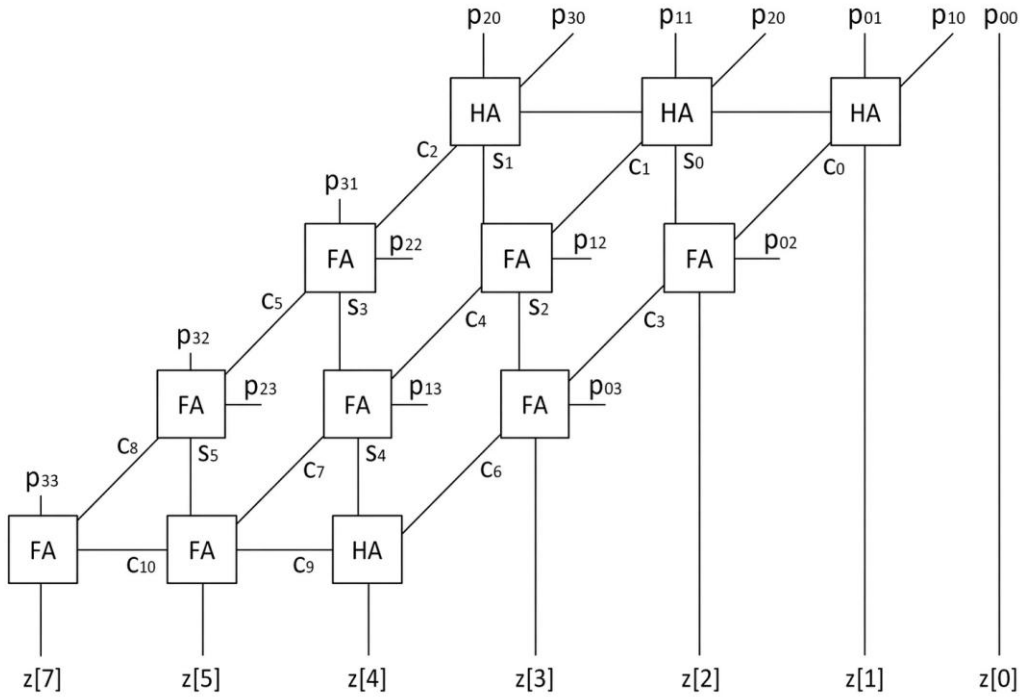

Figure 3.1: Array Multiplication Hardware Structure

3.2.2 Booth Encoding

Booth encoding is applied to reduce the number of partial products and thereby optimize timing, power, and area. As shown in Fig. 3.3, the Booth encoded hardware organizes the encoded partial products before accumulation.

3.2.2.1 *Radix-2 Encoding:*

In Radix-2 Booth encoding, the multiplier bits are examined in overlapping pairs (Qi, Qi-1). Each pair generates one partial product from the set {0, +M, -M} according to the encoding rules:

0←0 (right shift only), 1←1 (right shift only), 0←1 (add M then shift), and 1←0 (subtract M then shift). Table 3.1 illustrates the Radix-2 algorithm for the example 7 × 6 = 42.

Table 3.1: Booth Multiplier Example (7 × 6 = 42)

Step	A	Q	Q-1	Operation
Initial	0000	0110	0	Load values 0111
1) 0←0	0000 0000	0110 0011	0 0	Right shift only
2) 1←0	1001 1100	0011 1001	0 1	Perform A - M, Right shift
3) 1←1	1100 1110	1001 0100	1 1	Right shift only
4) 0←1	0101 0010	0100 1010	1 0	Perform A + M, Right shift
Final	00101010	—	—	Result = 42

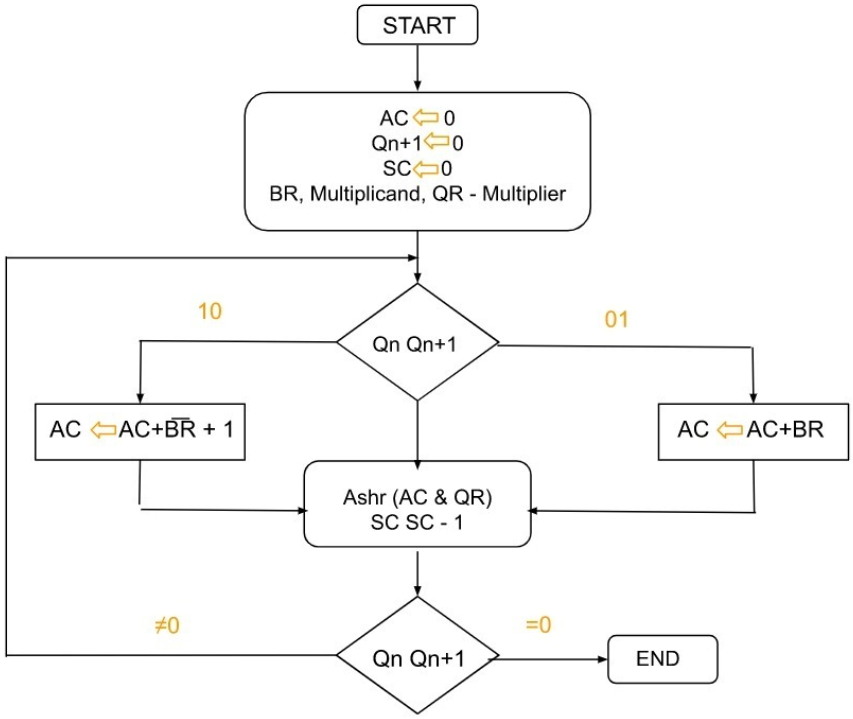

Figure 3.2: Booth Radix-2 Multiplier Flow Diagram

3.2.2.2 *Radix-4 Encoding:*

In Radix-4 Modified Booth encoding, overlapping groups of three multiplier bits (Q(i), Q(i-1), Q(i-2)) are examined simultaneously, generating partial products from the set {0, +M, -M, +2M, -2M}. This reduces the total number of partial products from n to n/2. Table 3.2 shows the complete encoding scheme. The sign bit is extended by one position to ensure an even number of bits, and the multiplier is appended with a zero at the LSB before encoding.

Table 3.2: Radix-4 Modified Booth Algorithm Encoding Scheme

Q(i)	Q(i-1)	Q(i-2)	Partial Product
0	0	0	+0
0	0	1	+M
0	1	0	+M
0	1	1	+2M
1	0	0	-2M
1	0	1	-M
1	1	0	-M
1	1	1	+0

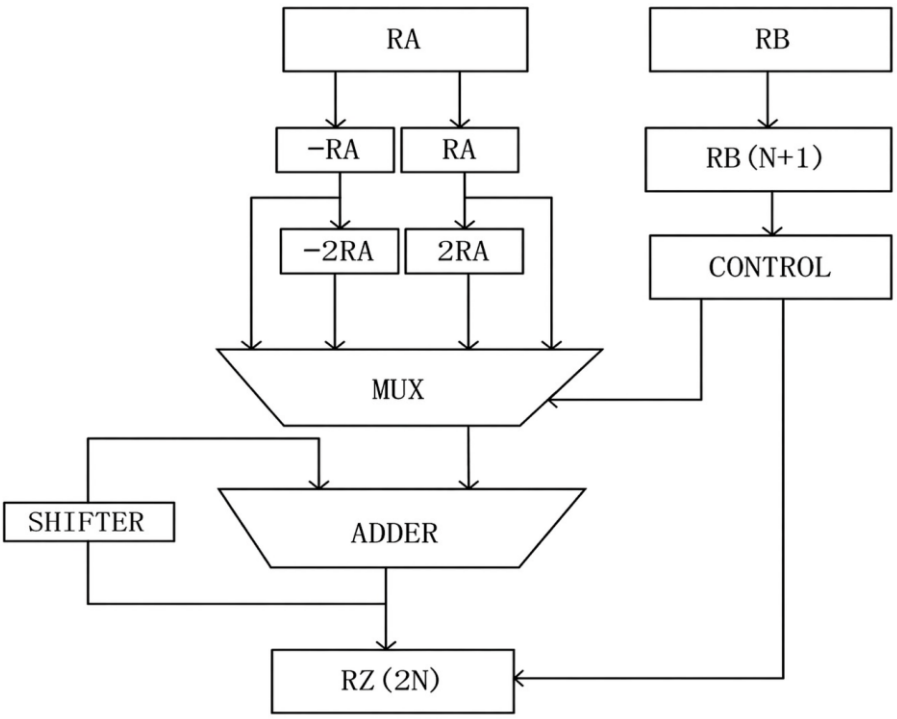

Figure 3.3: Booth Encoded Multiplication Hardware

3.2.3 Carry-Save Adder (CSA)

The Carry-Save Adder is used for accumulation of the partial products generated in the encoding stage. Unlike ripple-carry adders that propagate carries sequentially, the CSA adds three or more numbers simultaneously by deferring carry propagation. Each CSA stage reduces three partial products to two (a sum and a carry vector), enabling faster accumulation than sequential addition. The CSA was preferred over the Wallace Tree architecture in this project because the Wallace Tree's irregular structure results in significantly larger numbers of interconnection wires and routing complexity on FPGA targets, increasing chip design difficulty without a commensurate improvement at this bit-width.

3.2.4 Final Adder

The final adder takes the sum and carry outputs of the CSA network and produces the complete 2n-bit product. A ripple-carry adder is used at this stage, as the final addition involves only two operands and the carry propagation delay contributes to the overall critical path. The complete 16-bit result represents the signed product in 2's complement form.

4. EXPERIMENTS, RESULTS AND DISCUSSION

4.1 EXPERIMENTAL SETUP

All three multiplier architectures were implemented and evaluated on the Xilinx Basys-3 development board, which features the Artix-7 FPGA. The Vivado Design Suite was used for the complete design flow: RTL elaboration, synthesis, implementation (place and route), timing analysis, and power estimation. All designs were constrained identically, with a target clock frequency applied uniformly to ensure a fair comparison. Functional verification was performed using the Vivado XSim simulator with manually crafted testbenches covering both positive and negative operand combinations.

4.2 SIMULATION RESULTS

Functional simulation was performed for all three multiplier architectures. All designs produced correct 16-bit signed products for both positive and negative 8-bit operand combinations. As shown in Fig. 4.1, the Array multiplier simulation waveform confirms correct output for a range of test vectors. The Booth encoded multiplier simulation, depicted in Fig. 4.2, similarly verifies correct signed multiplication behavior under Radix-2 encoding. Both figures confirm that the structural Verilog implementations correctly execute the intended multiplication operations prior to synthesis.

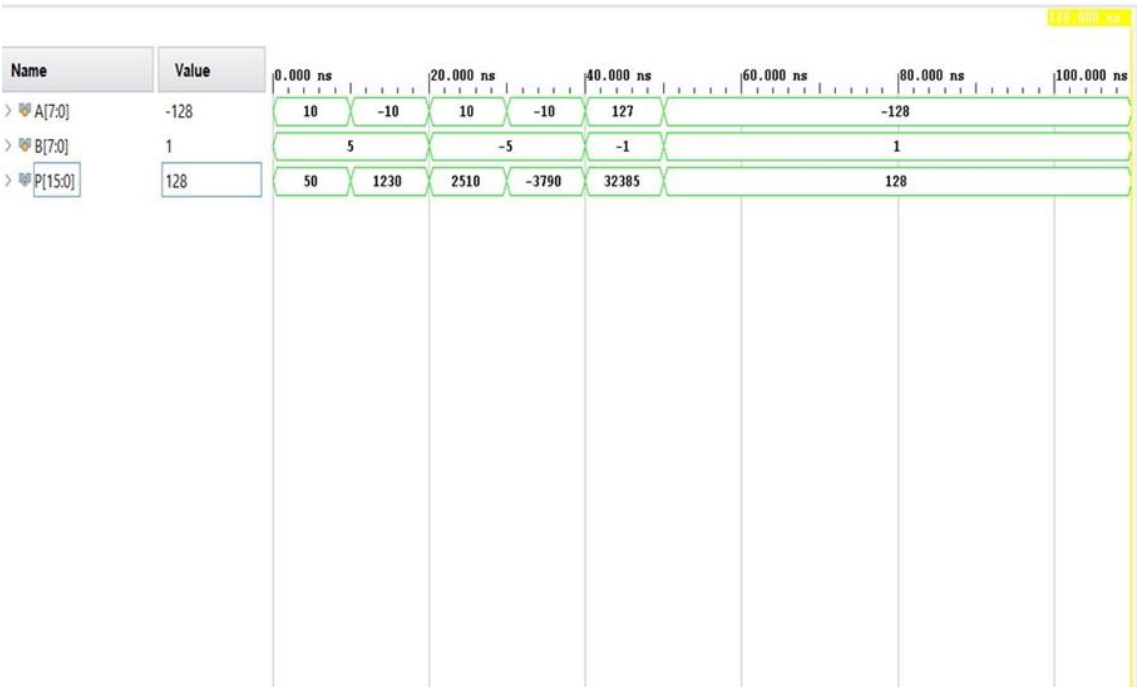

Figure 4.1: Array Multiplier Simulation Waveform

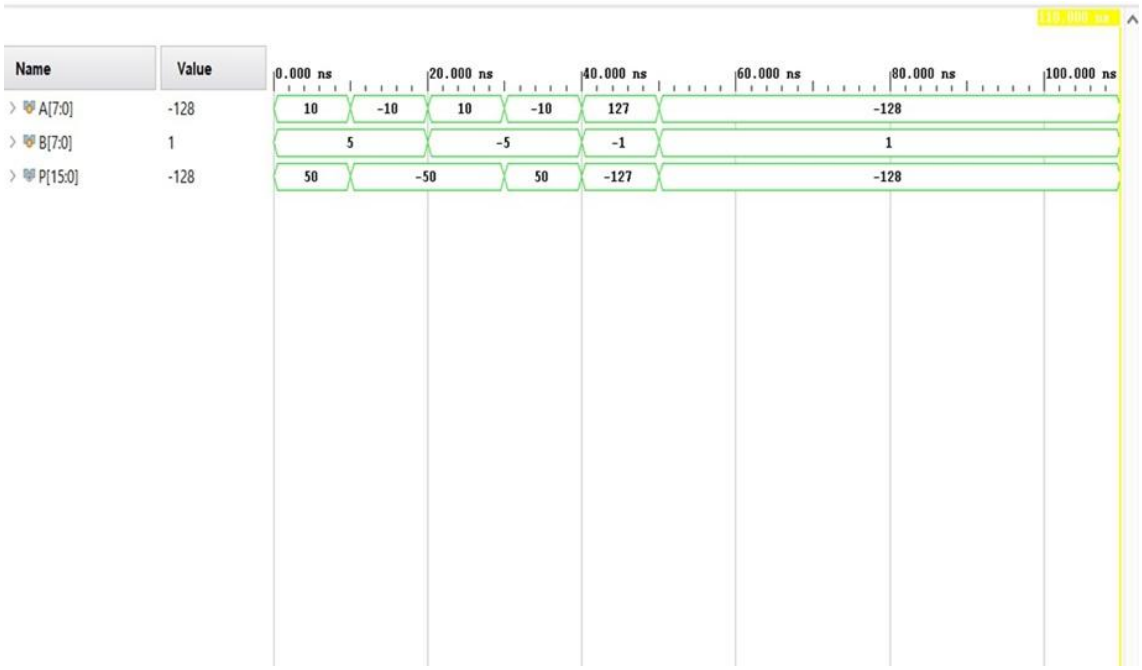

Figure 4.2: Booth Encoded Multiplier Simulation Waveform

4.3 LUT UTILIZATION

After synthesis and implementation, the LUT utilization for each architecture was extracted from Vivado's implementation report. As shown in Table 4.1 and Fig. 4.3, the Array multiplier achieves the most compact implementation with only 65 LUTs. This is because the AND-gate-based partial product generation maps efficiently onto FPGA LUT primitives with minimal overhead. Booth Radix-4 follows closely with 70 LUTs, benefiting from its reduced partial product count of n/2 achieved through 3-bit-at-a-time encoding. Booth Radix-2 consumes the most LUTs (124) because its per-partial-product encoder logic — which is applied to all n partial products — cannot be fully absorbed into the carry chain during synthesis, resulting in a larger combinational footprint.

Table 4.1: Comparison of 8-Bit Multipliers — LUTs, WHS, and WNS

Metric	Array Multiplier	Booth Radix-2	Booth Radix-4
No. of LUTs	65	124	70
WHS (ns)	0.308	0.265	0.271
WNS (ns)	3.332	2.326	3.009

Figure 4.3: LUT Utilization Comparison

4.4 TIMING ANALYSIS

Timing performance was evaluated using Worst Hold Slack (WHS) and Worst Negative Slack (WNS) extracted from Vivado's post-implementation timing reports. All three designs successfully met timing closure, as evidenced by positive WNS values across the board. The Array multiplier exhibits the largest WNS of 3.332 ns, indicating the greatest available timing margin and the simplest combinational path. Booth Radix-4 achieves a WNS of 3.009 ns, reflecting the shallower adder tree resulting from its reduced partial product count. Booth Radix-2 records the tightest WNS of 2.326 ns, due to the deeper combinational path introduced by its per-bit encoder and the larger partial product accumulation network. Figure 4.4 presents a graphical comparison of WHS and WNS values across the three architectures.

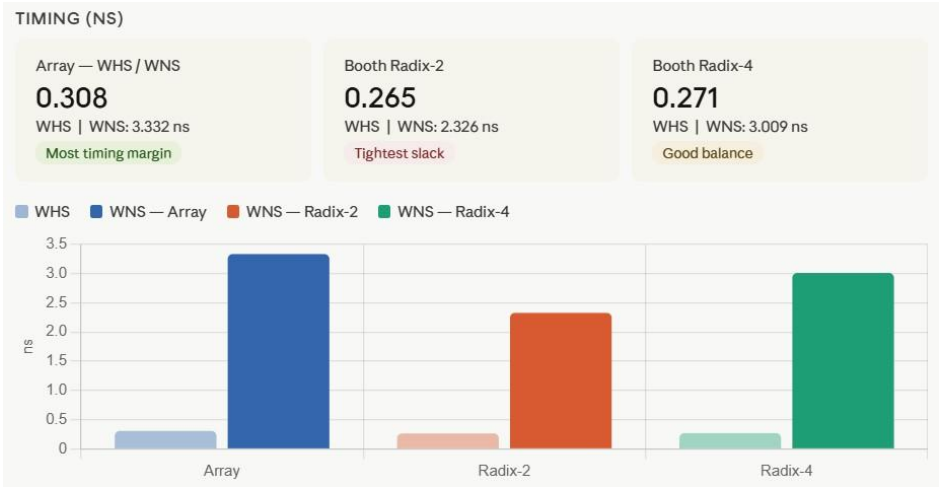

Figure 4.4: WHS/WNS Timing Comparison

4.5 POWER ANALYSIS

Power analysis was performed using Vivado's vectorless activity analysis, which provides medium-confidence estimates. As shown in Figures 4.5, 4.6, and 4.7, the total on-chip power is approximately 0.075 W, 0.073 W, and 0.076 W for the Array, Booth Radix-2, and Booth Radix-4 designs respectively. Device static power of 0.072 W dominates total consumption across all three designs, contributing 96–99% of total on-chip power. This is a fixed characteristic of the Artix-7 process technology and is unrelated to design activity.

Dynamic power values are 3 mW, 1 mW, and 5 mW for Array, Booth Radix-2, and Booth Radix-4 respectively. Contrary to theoretical expectations, Booth Radix-4 exhibits the highest dynamic power despite generating the fewest partial products. This anomaly is attributed to the ±2M operations in the Radix-4 encoder, which require additional shifting and sign-inversion logic, increasing node switching activity per clock cycle and outweighing the benefit of reduced partial product count at this bit-width. Booth Radix-2 achieves the lowest dynamic power due to its simpler 2-bit encoder producing sparser switching activity. Table 4.2 summarizes the power figures.

Table 4.2: Power Consumption Comparison Summary

Parameter	Array Multiplier	Booth Radix-2	Booth Radix-4
Total On-Chip Power (W)	0.075	0.073	0.076
Device Static Power (W)	0.072	0.072	0.072
Dynamic Power (mW)	3	1	5
Static Power Fraction (%)	96%	99%	94%

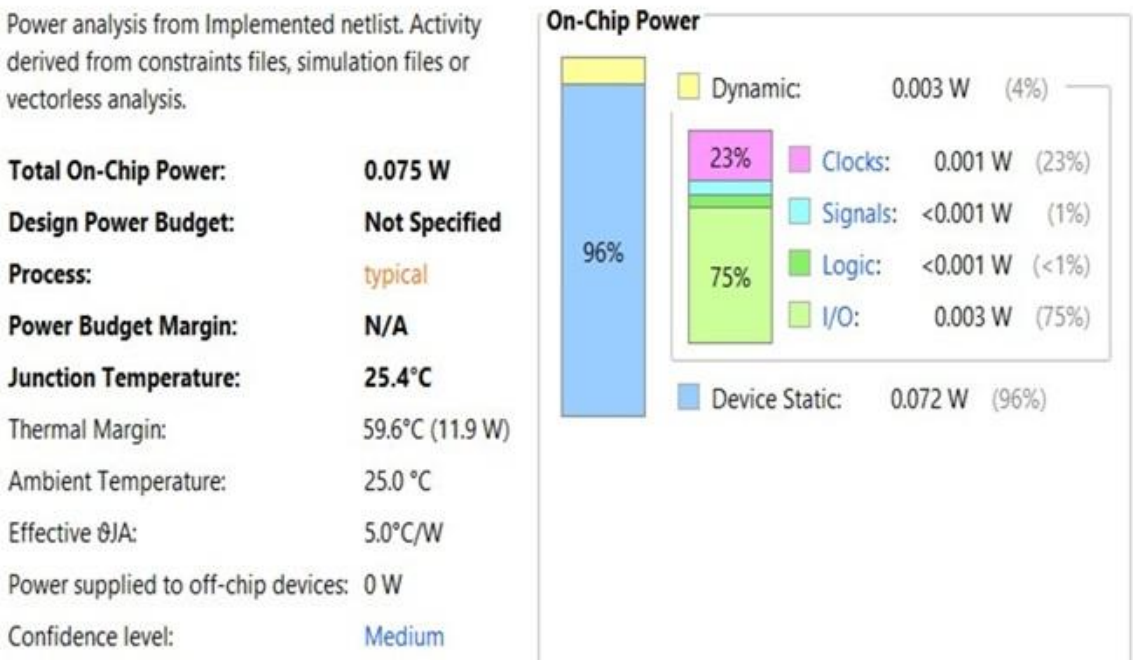

Figure 4.5: Power Report of Array Multiplier

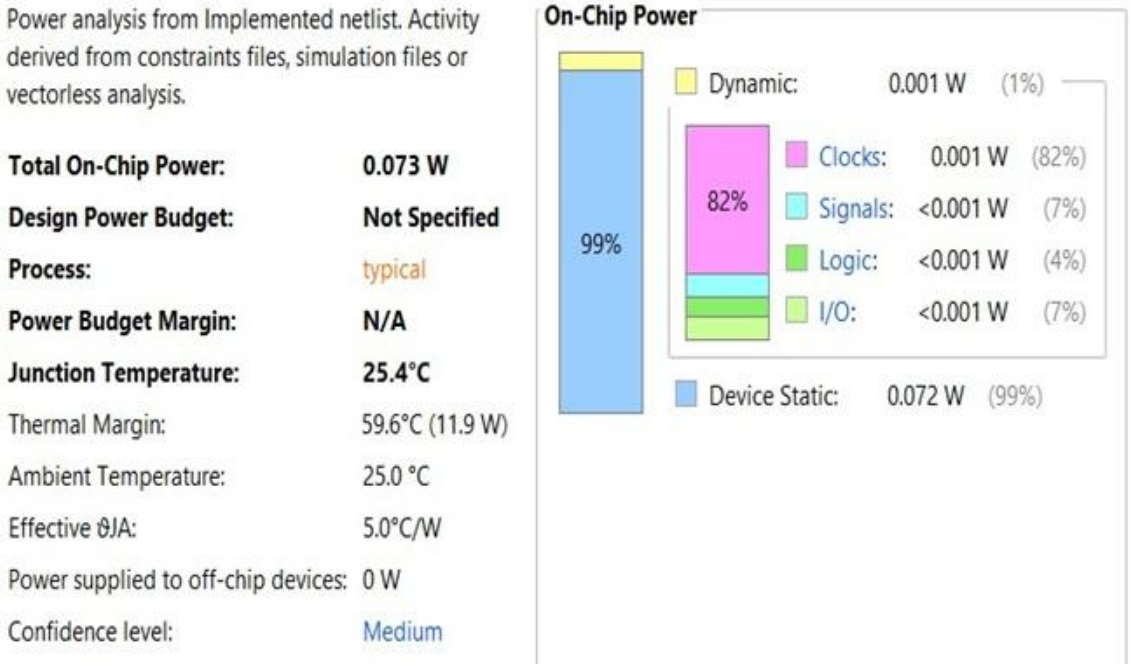

Figure 4.6: Power Report of Booth Radix-2 Multiplier

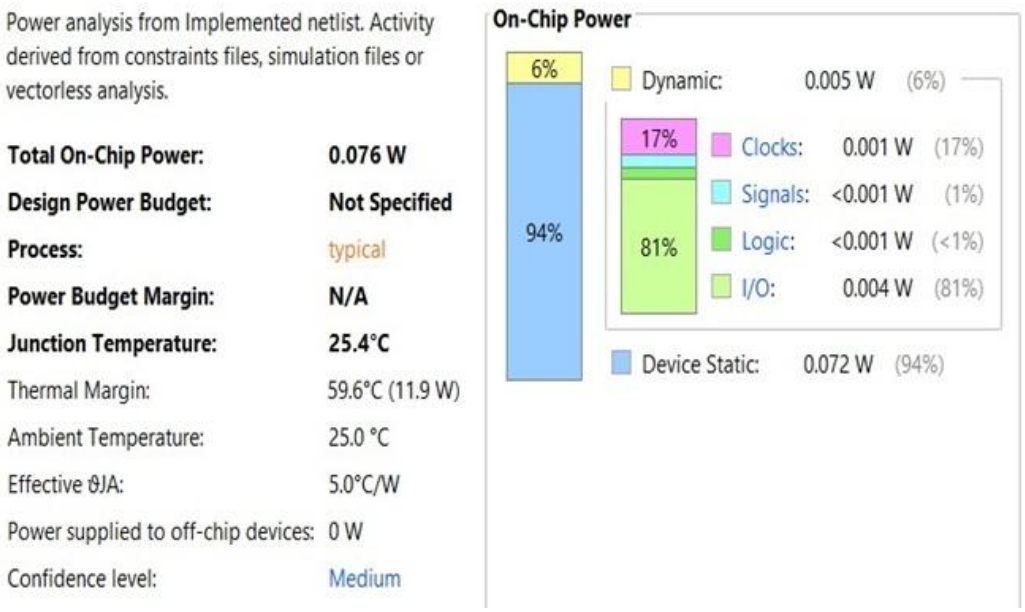

Figure 4.7: Power Report of Booth Radix-4 Multiplier

4.6 COMPARATIVE DISCUSSION

The comparative results across all 3 performances metrics reveal distinct trade-offs between the three architectures. No single design dominates across all dimensions. Table 4.3 presents a consolidated summary identifying the most suitable architecture for different design priorities.

Table 4.3: Comparative Summary — Best Architecture Per Metric

Metric	Array	Booth Radix-2	Booth Radix-4
LUTs (Area)	65 ✓ Best	124 — Worst	70 — Moderate
WNS (Timing Margin)	3.332 ns ✓ Best	2.326 ns — Tightest	3.009 ns — Good
Dynamic Power	3 mW — Moderate	1 mW ✓ Best	5 mW — Highest
Suitable For	Area-constrained designs	Power-critical systems	Speed-area balanced designs

These findings also indicate that gate-level benefits that exist theoretically may not necessarily manifest themselves in FPGA synthesis results. Technology mapping, logic utilization within LUTs, and static power consumption are just some of the factors that need to be taken into consideration when selecting a multiplier implementation.

5. HARDWARE IMPLEMENTATION

The proposed multiplier architectures were implemented and verified on the Digilent Basys3 development board, which houses the Xilinx Artix-7 (XC7A35T) FPGA. The board's onboard peripherals — 16 slide switches and 16 LEDs — were utilized as the input and output interfaces respectively, eliminating the need for any external hardware.

5.1 INTERFACE MAPPING

The 16 slide switches are mapped as follows: SW0–SW7 carry the 8-bit multiplicand (operand A) and SW8–SW15 carry the 8-bit multiplier (operand B). Both inputs are treated as signed 8-bit values in two's complement representation. The 16 LEDs (LD0–LD15) display the 16-bit product P, where LD0 represents the LSB and LD15 represents the MSB.

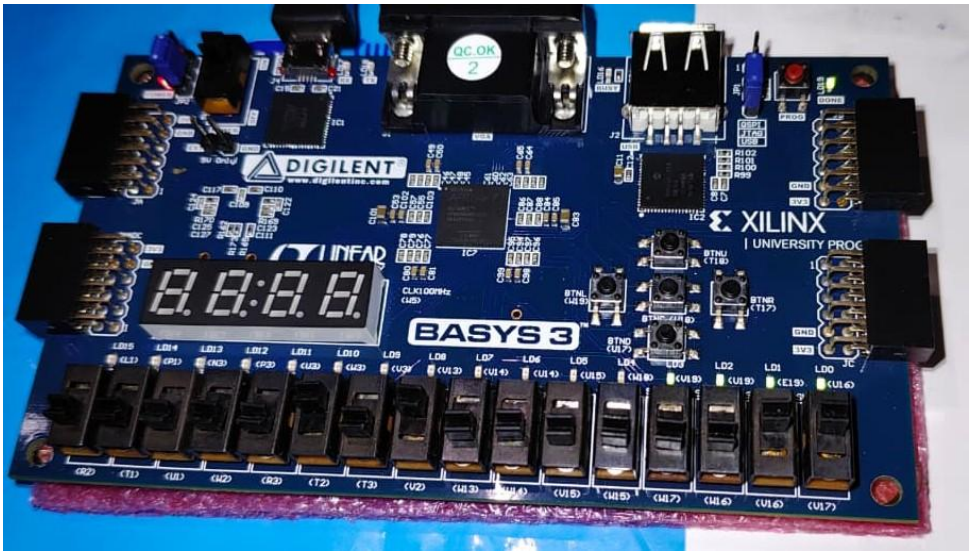

Figure 5.1: Basys 3 Board

5.2 TEST CASE 1: POSITIVE × POSITIVE

Multiplicand A = +3 (binary: 00000011)
Multiplier B = +5 (binary: 00000101).
Expected result: 3 × 5 = 15 → binary 0000000000001111.

Observed output: LD0, LD1, LD2, LD3 were HIGH and LD4–LD15 were LOW, corresponding to 0000000000001111 = 15 in decimal. The result confirms correct unsigned multiplication behaviour.

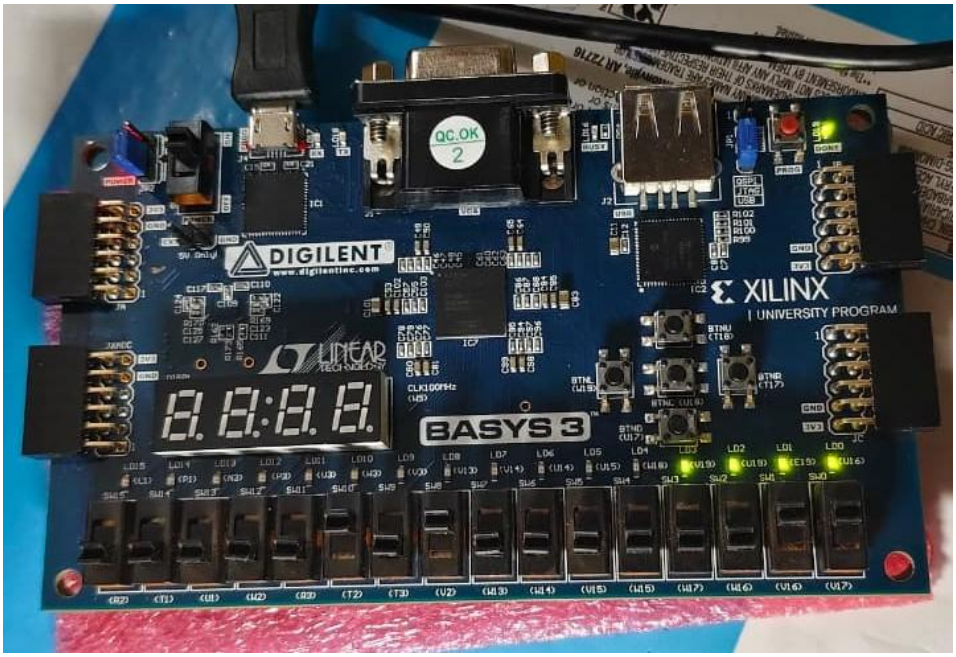

Figure 5.2: Result of Test Case 1

5.3 TEST CASE 2: NEGATIVE × POSITIVE

Multiplicand A = −3 (two's complement: 11111101)
Multiplier B = +5 (binary: 00000101).
Expected result: −3 × 5 = −15 → in 16-bit two's complement: 1111111111110001.

Observed output: LD0 was HIGH, LD1–LD3 were LOW, and LD4–LD15 were all HIGH, corresponding to 1111111111110001 which is the correct 16-bit two's complement representation of −15. This confirms that the signed multiplication and two's complement output are handled correctly by the hardware implementation.

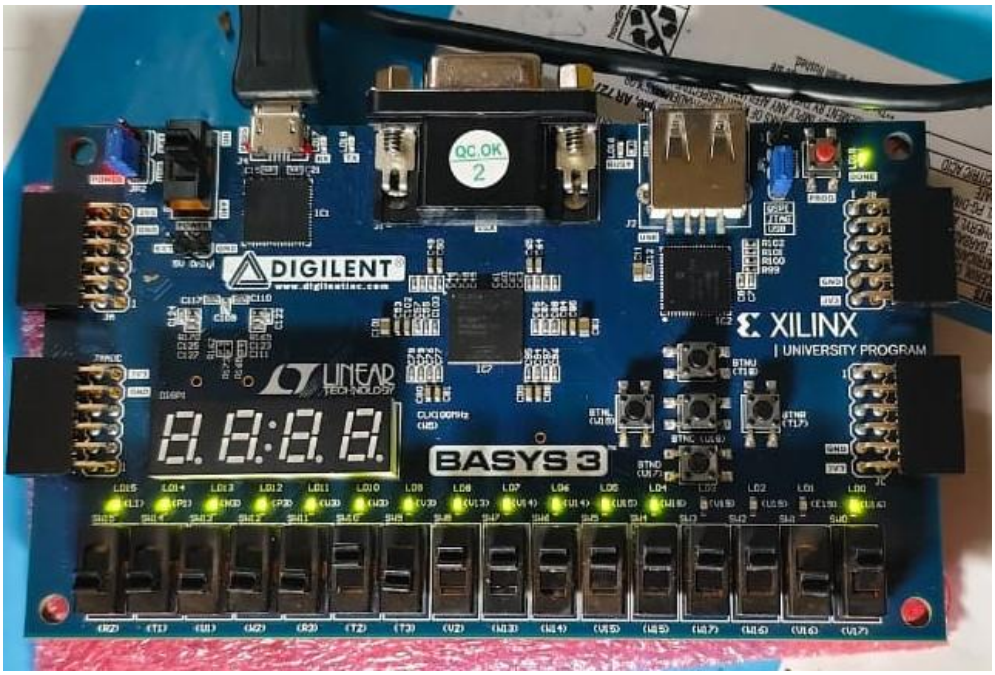

Figure 5.3: Result of Test Case 2

5.4 VERIFICATION SUMMARY

Both test cases produced results consistent with the expected theoretical values, validating the functional correctness of the synthesized design on physical hardware. The use of two's complement encoding for both input and output ensures compatibility with standard signed arithmetic conventions. The hardware verification confirms that the Vivado-generated bitstream faithfully implements the RTL description across all three multiplier architectures.

6. APPLICATIONS

The multiplier architectures studied in this project find broad application across numerous domains in digital electronics and computing. The choice of architecture depends on the specific constraints and requirements of the target application.

6.1 DIGITAL SIGNAL PROCESSING (DSP)

Multipliers are at the core of DSP operations including Finite Impulse Response (FIR) filtering, Fast Fourier Transform (FFT) computation, and convolution. High-throughput DSP systems benefit from Radix-4 architectures due to the reduced partial product count. The Array multiplier is preferred in resource-constrained embedded DSP modules.

6.2 MICROPROCESSORS AND ARITHMETIC LOGIC UNITS (ALUs)

Signed integer multiplication is a fundamental operation in general-purpose processor ALUs. Booth Radix-4 is widely adopted in modern processor data paths due to its favourable speed-area balance. Power-efficient processor designs for mobile and embedded platforms may favour Booth Radix-2 for its lower dynamic switching activity.

6.3 CRYPTOGRAPHIC HARDWARE

Modular multiplication forms the computational core of public-key cryptographic algorithms such as RSA and Elliptic Curve Cryptography (ECC). High-speed Radix-4 multipliers are commonly employed in hardware cryptographic accelerators where throughput is critical.

6.4 NEURAL NETWORK ACCELERATORS

Fixed-point multipliers are integral to hardware inference accelerators for deep neural networks. The ability to implement compact and power-efficient multipliers directly impacts the energy efficiency of edge AI devices. Array multipliers are well-suited for ultra-compact inference engines where silicon area is the primary constraint.

6.5 COMMUNICATIONS SYSTEMS

Multipliers are extensively used in channel equalization, modulation/demodulation, and coding algorithms in wireless communication systems. The low dynamic power of Booth Radix-2 makes it attractive for battery-powered communication devices.

6.6 EMBEDDED AND IOT SYSTEMS

Resource- and power-constrained IoT edge devices often require compact multiplier implementations. The Array multiplier, with its minimal LUT footprint and good timing margin, represents a practical choice for low-complexity embedded systems operating at moderate clock frequencies.

7. CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

In this project, three 8-bit signed multiplier architectures — Array, Booth Radix-2, and Booth Radix-4 — were designed in Verilog HDL and physically implemented on the Xilinx Artix-7 FPGA (Basys-3 board) using the Vivado Design Suite. All three designs were evaluated and compared across three key performance metrics: area utilization (LUT count), timing performance (WHS and WNS), and power consumption.

The Array multiplier achieved the most compact implementation with 65 LUTs and the largest timing margin (WNS = 3.332 ns), making it the preferred choice for area-constrained applications. Booth Radix-2, though consuming the most LUTs (124), recorded the lowest dynamic power (1 mW) owing to its simpler encoder and sparser switching activity, making it most suitable for power-critical designs. Booth Radix-4 presented a balanced trade-off with 70 LUTs and a WNS of 3.009 ns, though it unexpectedly exhibited the highest dynamic power (5 mW) due to the complexity of its modified encoder generating ±2M partial products.

A key finding of this work is that on FPGA platforms, technology mapping significantly influences implementation outcomes. Architectural advantages established at the theoretical gate level — such as the partial product reduction of Radix-4 — do not always manifest proportionally in FPGA synthesis due to LUT absorption characteristics, routing constraints, and the dominance of static device power. Designers must therefore validate multiplier selections through actual FPGA implementation rather than relying solely on algorithmic analysis.

7.2 FUTURE WORK

Several directions are proposed for extending this work:

- Extend the comparison to 16-bit and 32-bit operand widths to examine how area-speed-power trade-offs scale with increasing bit-width.
- Generate simulation-based switching activity files (SAIF) and annotate them into Vivado's power analysis flow for more accurate dynamic power estimation beyond the vector less activity model.

- Investigate ASIC implementation of the same three architectures using a standard cell library to compare FPGA-specific results with technology-independent gate-level performance.
- Develop pipelined variants of all three multipliers and evaluate throughput (operations per second) in addition to combinational delay.
- Explore Wallace Tree accumulation as an alternative to CSA accumulation and quantify the area-delay trade-off on FPGA at the same bit-width.
- Investigate the integration of these multipliers as MAC (Multiply-Accumulate) units in a simple DSP data path to evaluate system-level performance implications.

REFERENCES

1. [1] S. Churiwala, Designing with Xilinx® FPGAs Using Vivado. Springer, 2017.
2. [2] J. Cavanagh, Verilog HDL Design Examples. CRC Press, 2017.
3. [3] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, Digital Integrated Circuits: A Design Perspective, 2nd ed. Prentice Hall, 2003.
4. [4] B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs. Oxford University Press, 2000.
5. [5] N. Tam, L. Pham, and J. Benson, "Booth's Multiplier and 32-bit ALU for ARM7 Microprocessor," 2004.
6. [6] S. Shambhavi et al., "Multiplier Based on Add and Shift Method by Passing Zero," Dept. of Electronics and Communication, Sri Siddhartha Institute of Technology, 2014.
7. [7] "An Asynchronous, Iterative Implementation of the Original Booth Multiplication Algorithm," Dept. of Computer Science, University of Manchester.
8. [8] M. P. A., "Study of Multiplication Algorithms in Computer," 2014.
9. [9] B. Acharya, Computer Organisation and Architecture.
10. [10] A. D. Booth, "A Signed Binary Multiplication Technique," Quarterly Journal of Mechanics and Applied Mathematics, vol. 4, no. 2, pp. 236 240, 1951.
11. [11] C. R. Baugh and B. A. Wooley, "A Two's Complement Parallel Array Multiplication Algorithm," IEEE Transactions on Computers, vol. C-22, no. 12, pp. 1045–1047, 1973.
12. [12] C. S. Wallace, "A Suggestion for a Fast Multiplier," IEEE Transactions on Electronic Computers, vol. EC-13, pp. 14–17, 1964.
13. [13] "Faster Implementation of Booth's Algorithm Using FPGA," IEEE International Conference, 2017.

14. [14] “Usage Area and Speed Performance Analysis of Booth Multiplier on Its FPGA Implementation,” IEEE Conference Publication, 2017.

15. [15] “FPGA Implementation of Single Cycle Signed Multiplier Using Booth Recoding Algorithm,” Academia, 2023.

16. [16] “Hybrid Modified Booth Encoded Algorithm–Carry Save Adder Fast Multiplier,” IEEE Conference Publication, 2015.

17. [17] Jia et al., “Simplified Carry Save Adder-Based Array Multiplier Scheme and Circuits Design,” International Journal of Circuit Theory and Applications, 2015.

18. [18] A. Parsodia and P. Jindal, “Design of Low Power Booth Multiplier with Enhanced Pre-logic Mechanism Using Verilog,” in Smart Trends in Computing and Communications (SmartCom 2024), Springer, 2024.

19. [19] C. M. Kalaiselvi and R. S. Sabeenian, “A Modular Technique of Booth Encoding and Vedic Multiplier for Low-Area and High-Speed Applications,” Scientific Reports, 2023.

20. [20] “Comparison of Array Multiplier, Wallace Tree Multiplier and Booth Multiplier on Xilinx FPGA,” International Journal of Computer, vol. 2, no. 4, 2010.

21. [21] D. E. Thomas and P. R. Moorby, The Verilog Hardware Description Language, 5th ed. Springer, 2002.

22. [22] Digilent Inc., Basys 3 Artix-7 FPGA Trainer Board Reference Manual, 2016.

PUBLICATIONS

Paper is ready to submit in ICECSP 2026 - 2nd International Conference on Electronics, Communication and Signal Processing, Organized by Department of Electronics and Communication Engineering, NIT Delhi.